

## 1 Introdução

Neste projeto, seu agente Pacman deve encontrar caminhos através de dos labirintos, de modo a cumprir algumas tarefas, como chegar em algum local específico ou coletar comida de forma eficiente. Sua tarefa sera desenvolver algoritmos de busca e aplicá-los nos diversos cenários do Pacman.

## 2 Requisitos Administrativos

O trabalho deve ser realizado em duplas ou trios. **Não serão aceitos trabalhos individuais.**

A data da entrega será [02/setembro](#) pelo moodle. Um membro do grupo deverá enviar o arquivo contendo o código desenvolvido e o relatório, conforme descrito a seguir. A nota deste trabalho será parte da média de trabalhos práticos (TP) da disciplina.

## 3 Código Base

O código base do projeto que utilizaremos para esse trabalho foi desenvolvido por uma equipe da Universidade de Berkeley, na Califórnia. Este projeto consiste em diversos arquivos Python, alguns dos quais você precisará compreender para completar as tarefas, e alguns que você pode ignorar.

Arquivos que você irá editar:

- `search.py` – Aqui serão implementados os algoritmos de busca
- `searchAgents.py` – Esse arquivo vai conter os agentes baseados em busca, que você deve desenvolver

Arquivos que você deve estudar (mas não pode editar!):

- `pacman.py` – Roda o jogo. Este arquivo contem a classe `GameState` que será a usada neste projeto
- `game.py` – Implementa a logica do mundo do Pacman. Possui diversas classes de interesse, como `AgentState`, `Agent`, `Direction` e `Grid`.
- `util.py` Contém a implementação de estruturas de dados úteis para algoritmos de busca (Pilha, Fila e Fila de Prioridades)

Arquivos de suporte que você pode ignorar:

- `graphicsDisplay.py` – Gráficos do Pacman
- `graphicsUtils.py` – Funções de suporte ao jogo
- `textDisplay.py` – Gráficos ASCII para o Pacman

- `ghostAgents.py` – Agentes que controlam os fantasmas
- `keyboardAgents.py` – Interfaces de controle do jogo pelo teclado
- `layout.py` – Código para ler arquivos de layout e guardar seu conteúdo
- `autograder.py` – Roda testes automáticos no projeto
- `testParser.py` – Parser para testes do autograder e arquivos de solução
- `testClasses.py` – Classe de testes do autograder
- `test cases/` – Diretório que contém os casos de teste para cada problema
- `searchTestClasses.py` – Classes de teste do Projeto 1

## 4 Ambiente de desenvolvimento

Depois de descompactar o código deste projeto, você pode executar o programa utilizando o seguinte comando:

```
python pacman.py
```

Este comando executa uma versão do jogo em que você controla o Pacman utilizando as setas do teclado.

O agente mais simples no arquivo `searchAgents.py` é o agente `GoWestAgent`, que sempre se move para o oeste (ele é um agente reativo simples). Ele consegue vencer em uma configuração simples do mundo:

```
python pacman.py --layout testMaze --pacman GoWestAgent
```

Porém, com um layout de labirinto em que é necessário fazer curvas, as coisas já não funcionam tão bem:

```
python pacman.py --layout tinyMaze --pacman GoWestAgent
```

A execução de `pacman.py` permite diversas opções que podem ser expressas de forma extensa (exemplo: `{layout}`), ou de forma reduzida (exemplo: `-l`). A lista com todas as opções e valores padrão pode ser vista utilizando o comando `help`:

```
python pacman.py -h
```

O arquivo `commands.txt` contém todos os comandos que estão na descrição deste trabalho, para facilitar o processo de copiar/colar.

## 5 Tarefas

No arquivo `searchAgents.py`, você encontrará a classe `SearchAgent` completamente implementada, que planeja um caminho através do mundo do Pacman e depois executa o caminho passo a passo. Os algoritmos de busca que serão usados para formular este plano ainda não estão implementados.

Para testar se o `SearchAgent` está funcionando, você pode executar o comando abaixo:

```
python pacman.py -l tinyMaze -p SearchAgent -a fn=tinyMazeSearch
```

O comando acima diz para o `SearchAgent` utilizar como algoritmo de busca a função `tinyMazeSearch`, que está implementada em `search.py`. Para este labirinto, o Pacman deve conseguir navegar pelo mundo sem problemas. Note que `tinyMazeSearch` retorna uma lista de ações!

**Importante:** Todas as funções de busca devem retornar uma lista de ações que levarão o agente do estado inicial até o estado objetivo. Estas ações precisam ser movimentos válidos dentro do jogo (nomes válidos, não atravessar as paredes, etc.).

*Dica 1:* O tabuleiro do Pacman mostra um overlay dos estados explorados e a ordem em que eles foram explorados pelo algoritmo de busca utilizado. Quanto mais forte o tom de vermelho, mais cedo o estado foi explorado. Você pode verificar se o seu algoritmo de busca está funcionando como o previsto analisando visualmente o padrão de exploração de estados.

*Dica 2:* A implementação dos algoritmos é muito parecida. Os algoritmos BEP, BEL e BCU diferem apenas nos detalhes de como a fronteira é gerenciada. Portanto, concentre-se em fazer um deles corretamente e o restante deve ser relativamente simples. Na verdade, é possível resolver esse trabalho com um único método de busca genérico que é configurado com uma estratégia de fila específica do algoritmo.

## 5.1 Tarefa 1: Busca em profundidade (Depth First Search)

Implemente o algoritmo de busca em profundidade (BEP) na função `depthFirstSearch` no arquivo `search.py`. Para tornar seu algoritmo completo, escreva a versão de busca em grafo do BEP, que evita expandir estados já visitados. Sua implementação deve funcionar para todos os layouts de labirinto abaixo:

```
python pacman.py -l tinyMaze -p SearchAgent -a fn=dfs
```

```
python pacman.py -l mediumMaze -p SearchAgent -a fn=dfs
```

```
python pacman.py -l bigMaze -z .5 -p SearchAgent -a fn=dfs
```

Utilize a chamada abaixo para verificar se seu código passa em todos os testes automáticos do autograder:

```
python autograder.py -q q1
```

## 5.2 Tarefa 2: Busca em largura (Breadth First Search)

Implemente um algoritmo de busca em largura na função `breadthFirstSearch` dentro do arquivo `search.py`. A versão implementada deve ser da busca em grafo, ou seja, evite explorar nodos da fronteira que já tenham sido visitados. Para testar sua implementação, utilize os comandos abaixo:

```
python pacman.py -l tinyMaze -p SearchAgent -a fn=bfs
```

```
python pacman.py -l mediumMaze -p SearchAgent -a fn=bfs
```

```
python pacman.py -l bigMaze -z .5 -p SearchAgent -a fn=bfs
```

*Observação:* Nosso método de busca é escrito de forma tão genérica que você deve conseguir rodar a busca em largura para resolver o problema 8-puzzle sem realizar alterações no seu código:

```
python eightpuzzle.py
```

Utilize a chamada abaixo para verificar se seu código passa em todos os testes:

```
python autograder.py -q q2
```

### 5.3 Tarefa 3: Busca de custo uniforme (Uniform Cost Search)

Enquanto a busca em largura encontra o menor caminho até o objetivo, pode ser interessante encontrar caminhos que são “melhores” em outros sentidos. Por exemplo, podemos penalizar nosso agente por andar em áreas infestadas de fantasmas ou recompensá-lo por passos em áreas ricas em alimentos, e um agente Pacman racional deve ajustar o seu comportamento em resposta.

Implemente um algoritmo de busca de custo uniforme em grafos na função `uniformCostSearch` no arquivo `search.py`. Seu algoritmo deve ter sucesso em todos os layouts de labirinto dos comandos abaixo, onde custos de ações variam (as funções de custo já estão implementadas).

```
python pacman.py -l mediumMaze -p SearchAgent -a fn=ucs
```

```
python pacman.py -l mediumDottedMaze -p StayEastSearchAgent
```

```
python pacman.py -l mediumScaryMaze -p StayWestSearchAgent
```

Utilize a chamada abaixo para verificar se seu código passa em todos os testes:

```
python autograder.py -q q3
```

### 5.4 Tarefa 4: Busca A\*

Implemente um algoritmo de busca A\* em grafos na função `aStarSearch` no arquivo `search.py`. Esta função requer uma função heurística como argumento. Uma função heurística possui dois argumentos: um estado no problema de busca e o próprio problema. A função `nullHeuristic` em `search.py` é um exemplo trivial implementado em que todos os custos de heurística são nulos. Já a função `manhattanHeuristic` em `searchAgents.py` é uma heurística que computa a distância de Manhattan.

Para testar sua implementação do algoritmo A\* usando a distância de Manhattan como função heurística, utilize o comando abaixo:

```
python pacman.py -l bigMaze -z .5 -p SearchAgent -a fn=astar,heuristic=manhattanHeuristic
```

Utilize a chamada abaixo para verificar se seu código passa em todos os testes:

```
python autograder.py -q q4
```

### 5.5 Tarefa 5: O problema dos cantos

O verdadeiro poder do A\* só se tornará evidente diante de um problema de busca mais desafiador. Agora é o momento de formular um novo problema e projetar uma heurística para ele.

Em labirintos de cantos (\*corner mazes\*), há quatro pontos de comida, um em cada canto. Nosso novo problema de busca consiste em encontrar o caminho mais curto pelo labirinto que passe por todos os quatro cantos (independentemente de o labirinto realmente conter comida nesses locais). Observe que, para alguns labirintos como `tinyCorners`, o caminho mais curto nem sempre visita primeiro o alimento mais próximo! Dica: o menor caminho em `tinyCorners` possui 28 passos.

*Observação:* certifique-se de concluir a Tarefa 2 antes de trabalhar na Tarefa 5, pois a Tarefa 5 se baseia na sua resposta da Tarefa 2.

Implemente os métodos da classe `CornersProblem` (que estende a classe `SearchProblem`) no arquivo `searchAgents.py`. Será necessário escolher uma representação de estado que codifique todas as informações necessárias para detectar se os quatro cantos já foram alcançados. Assim, seu agente de busca deverá resolver:

```
python pacman.py -l tinyCorners -p SearchAgent -a fn=bfs,prob=CornersProblem
```

```
python pacman.py -l mediumCorners -p SearchAgent -a fn=bfs,prob=CornersProblem
```

Tome cuidado para que a representação abstrata de estado proposta não codifique informações irrelevantes (como a posição dos fantasmas, a localização de alimentos extras etc.). Em particular, não utilize um `GameState` do Pacman como estado de busca. Caso contrário, seu código será extremamente lento (e também incorreto).

Uma instância da classe `CornersProblem` representa todo o problema de busca, e não um estado específico. Estados particulares são retornados pelas funções que você escrever, e essas funções devem retornar uma estrutura de dados à sua escolha (por exemplo, `tuple`, `set`, etc.) que represente um estado.

Além disso, durante a execução do programa, muitos estados existem simultaneamente, todos na fila do algoritmo de busca, e eles devem ser independentes entre si. Em outras palavras, não deve existir apenas um único estado para todo o objeto `CornersProblem`; sua classe deve ser capaz de gerar vários estados diferentes para fornecer ao algoritmo de busca.

*Dica 1:* as únicas partes do estado do jogo que precisam ser referenciadas em sua implementação são a posição inicial do Pacman e a localização dos quatro cantos.

*Dica 2:* ao implementar `getSuccessors`, certifique-se de adicionar os filhos à lista de sucessores com custo igual a 1.

Nossa implementação de `breadthFirstSearch` expande pouco menos de 2000 nós de busca em `mediumCorners`. Entretanto, heurísticas (utilizadas com a busca A\*) podem reduzir a quantidade de exploração necessária.

**Avaliação:** execute o comando abaixo para verificar se sua implementação passa em todos os testes do autograder.

```
python autograder.py -q q5
```

## 5.6 Tarefa 6: O problema dos cantos (com heurísticas)

*Observação:* certifique-se de concluir a Tarefa 4 antes de trabalhar na Tarefa 6, pois a Tarefa 6 se baseia na sua resposta da Tarefa 4.

Implemente uma heurística não trivial para o `CornersProblem` em `cornersHeuristic`.

```
python pacman.py -l mediumCorners -p AStarCornersAgent -z 0.5
```

Observação: `AStarCornersAgent` é um atalho para

```
-p SearchAgent -a fn=aStarSearch,prob=CornersProblem,heuristic=cornersHeuristic
```

*Admissibilidade:* lembre-se de que heurísticas são apenas funções que recebem estados de busca e retornam valores que estimam o custo até um objetivo mais próximo. Heurísticas mais eficazes retornam valores mais próximos dos custos reais. Para ser admissível, os valores da heurística devem ser *otimistas*, ou seja, seu valor deve ser igual ou inferior ao custo real do menor caminho até o objetivo mais próximo (e não negativos).

*Heurísticas não triviais:* as heurísticas triviais são aquelas que retornam zero em todos os casos (equivalente à BCU) e a heurística que retorna o custo real da solução. A primeira não economiza tempo, enquanto a segunda fará o autograder exceder o tempo limite. O objetivo é uma heurística que reduza o tempo total de computação; nesta tarefa, entretanto, o autograder verificará apenas a contagem de nós (além de impor um limite de tempo razoável).

*Avaliação:* sua heurística deve ser não trivial e não negativa para receber qualquer pontuação. Certifique-se de que ela retorna 0 em todo estado objetivo e nunca retorna valores negativos. Dependendo do número de nós expandidos:

Execute:

| Número de nós expandidos | Nota  |
|--------------------------|-------|
| mais de 2000             | 0 / 3 |
| no máximo 2000 nodos     | 1 / 3 |
| no máximo 1600 nodos     | 2 / 3 |
| no máximo 1200 nodos     | 3 / 3 |

```
python autograder.py -q q6
```

## 5.7 Tarefa 7: Comendo toda a comida

Agora resolveremos um problema de busca mais difícil: consumir toda a comida do Pacman no menor número possível de passos. Para isso, será necessária uma nova definição de problema de busca que leva em conta a presença de comida no labirinto: `FoodSearchProblem` em `searchAgents.py` (já implementado).

Uma solução é definida como um caminho que coleta toda a comida no mundo do Pacman. Neste projeto, as soluções não consideram fantasmas nem power pellets (pontos grandes). Elas dependem apenas da disposição dos muros, da comida e do Pacman. Se seus métodos gerais de busca estiverem corretos, o A\* com heurística nula (equivalente à busca de custo uniforme) deverá encontrar rapidamente uma solução ótima para `testSearch`, sem necessidade de alterar código (custo total 7).

```
python pacman.py -l testSearch -p AStarFoodSearchAgent
```

*Observação:* `AStarFoodSearchAgent` é um atalho para

```
-p SearchAgent -a fn=astar,prob=FoodSearchProblem,heuristic=foodHeuristic
```

Você perceberá que a BCU começa a ficar lenta mesmo para o labirinto simples `tinySearch`. Como referência, nossa implementação leva 2,5 segundos para encontrar um caminho de comprimento 27 após expandir 5057 nós de busca.

*Observação:* conclua a Tarefa 4 antes de trabalhar na Tarefa 7, pois a Tarefa 7 depende dela.

Preencha `foodHeuristic` em `searchAgents.py` com uma heurística para o `FoodSearchProblem`. Teste seu agente no tabuleiro `trickySearch`:

```
python pacman.py -l trickySearch -p AStarFoodSearchAgent
```

Nosso agente BCU encontra a solução ótima em aproximadamente 13 segundos, explorando mais de 16.000 nós.

Qualquer heurística não trivial e não negativa receberá 1 ponto. Certifique-se de que a heurística retorna 0 em todo estado objetivo e nunca retorna valores negativos. Pontuação adicional conforme a redução de nós:

| Número de nós expandidos | Nota                                  |
|--------------------------|---------------------------------------|
| mais de 15.000           | 1 / 4                                 |
| no máximo 15.000         | 2 / 4                                 |
| no máximo 12.000         | 3 / 4                                 |
| no máximo 9.000          | 4 / 4 (pontuação máxima; médio)       |
| no máximo 7.000          | 5/4 (crédito extra opcional; difícil) |

Execute o comando abaixo para avaliar sua solução:

```
python autograder.py -q q7
```

## 5.8 Tarefa 8: Busca Subótima

Às vezes, mesmo utilizando o algoritmo A\* com uma boa heurística, encontrar o caminho ótimo que percorra todos os pontos é difícil. Nesses casos, ainda desejamos encontrar rapidamente um caminho razoavelmente bom. Nesta tarefa, você escreverá um agente que sempre consome, de forma gulosa, o ponto mais próximo. `ClosestDotSearchAgent` já está implementado em `searchAgents.py`, mas falta uma função essencial que encontra um caminho até o ponto mais próximo.

Implemente a função `findPathToClosestDot` em `searchAgents.py`. Nosso agente resolve este labirinto (de forma subótima!) em menos de um segundo, com custo de caminho igual a 350:

```
python pacman.py -l bigSearch -p ClosestDotSearchAgent -z .5
```

*Dica:* a maneira mais rápida de completar `findPathToClosestDot` é implementar `AnyFoodSearchProblem`, ao qual falta apenas o teste de objetivo. Em seguida, resolva esse problema com uma função de busca apropriada. A solução deve ser bastante curta.

O agente `ClosestDotSearchAgent` nem sempre encontrará o menor caminho possível pelo labirinto. Certifique-se de compreender o motivo e tente elaborar um pequeno exemplo no qual visitar repetidamente o ponto mais próximo não produz o caminho mínimo para consumir todos os pontos.

*Avaliação:* execute o autograder para verificar se sua implementação passa em todos os testes.

```
python autograder.py -q q8
```

## 6 Relatório

Cada grupo deve entregar, junto com o código-fonte, um relatório escrito em **formato Markdown** (arquivo `relatorio.md`, incluído no mesmo arquivo enviado pelo Moodle). O relatório deve ser objetivo e deve descrever principalmente as funções heurísticas implementadas nas Tarefas 6 e 7, além de um resumo geral da solução. Utilize a seguinte estrutura como guia:

1. **Identificação do grupo:** nomes e cartões dos integrantes.
2. **Visão geral da implementação:** breve descrição de como o método de busca genérico (usado nas Tarefas 1 a 4) foi estruturado
3. **Representação de estado do `CornersProblem` (Tarefa 5):** qual estrutura de dados foi escolhida para representar um estado e por que ela é suficiente e mínima para o problema.
4. **Heurísticas implementadas:** para cada heurística abaixo, descreva:
  - o raciocínio por trás da heurística (que informação do estado ela usa e como estima o custo restante);
  - a fórmula ou pseudocódigo utilizado no cálculo;
  - a justificativa de **admissibilidade** (por que o valor retornado nunca supera o custo real até o objetivo mais próximo) e, se aplicável, de **consistência**;
  - os resultados obtidos (número de nós expandidos, tempo de execução e custo do caminho encontrado) nos labirintos de teste indicados no enunciado, comparados aos valores de referência fornecidos.

As duas heurísticas que devem ser detalhadas são:

- `cornersHeuristic` (Tarefa 6, problema `CornersProblem`) – ex.: distância de Manhattan até o canto não visitado mais próximo, soma/combinção das distâncias entre os cantos restantes, etc.;

- **foodHeuristic** (Tarefa 7, problema **FoodSearchProblem**) – ex.: maior distância de Manhattan até a comida restante, uso de distância real de labirinto (via BFS) entre pares de pontos de comida, técnicas de memoização/cache utilizadas para acelerar o cálculo, etc.
5. **Uso de ferramentas de IA:** Declaração do grupo de quais ferramentas de IA foram utilizadas no desenvolvimento do trabalho e para quais tarefas.